МИНОБРНАУКИ РОССИИ

САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ

ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ

«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)

Кафедра Робототехники и автоматизации производственных систем

ОТЧЕТ

по лабораторной работе № 3

по дисциплине «Техническое зрение и специальное программное обеспечение»

ТЕМА: ПОИСК ГРАНИЦ НА ИЗОБРАЖЕНИЯХ.

Студент гр. 1406 _____ Бондаренко Ю.В.

Преподаватель _____ Федоркова А.О.

Санкт-Петербург

2026

Цель работы – изучить методы дифференцирования изображений и выделения границ на изображениях.

Задание №1

Операторы Собеля хорошо определяют границы по соответствующим осям, однако в итоговых изображениях присутствует зашумление (рисунки 2, 3, 6, 7, 10, 11). Операторы Лапласа, в свою очередь, используют суммарный градиент и качественно определяют границы во всех направлениях (рисунки 4, 8, 12).

Следующий код одинаков для всех рассматриваемых изображений, меняется лишь путь к исходному файлу:

```
path = r"./3-1.jpg"
picture = cv2.imread(path, cv2.IMREAD_REDUCED_GRAYSCALE_8)
sobelx = cv2.Sobel(picture, cv2.CV_64F, 1, 0, ksize=11) #Градиент по X
sobely = cv2.Sobel(picture, cv2.CV_64F, 0, 1, ksize=5) #Градиент по Y
laplace = cv2.Laplacian(picture, -1, ksize=3)
cv2.imshow("3-1.jpg",picture)
cv2.imshow("x",sobelx)
cv2.imshow("y",sobely)
cv2.imshow("laplace",laplace)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

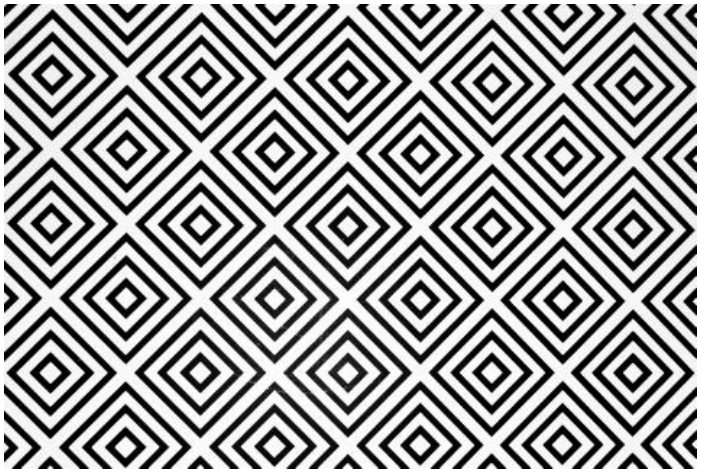

Рисунок 1 – Исходное изображение №1

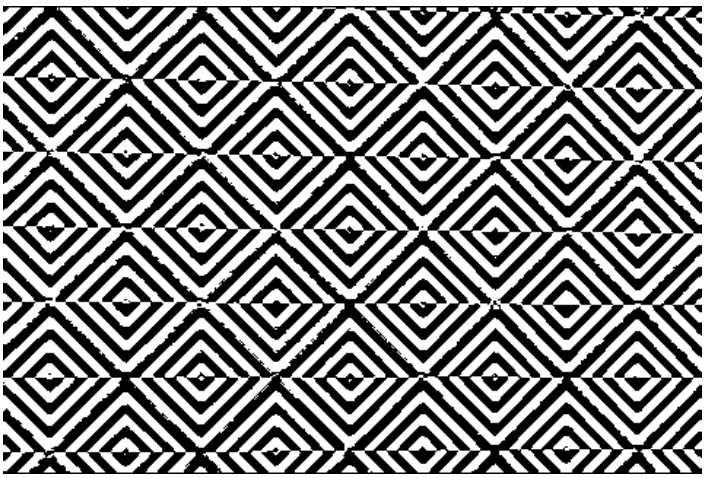

Рисунок 2 – Градиент по оси Y

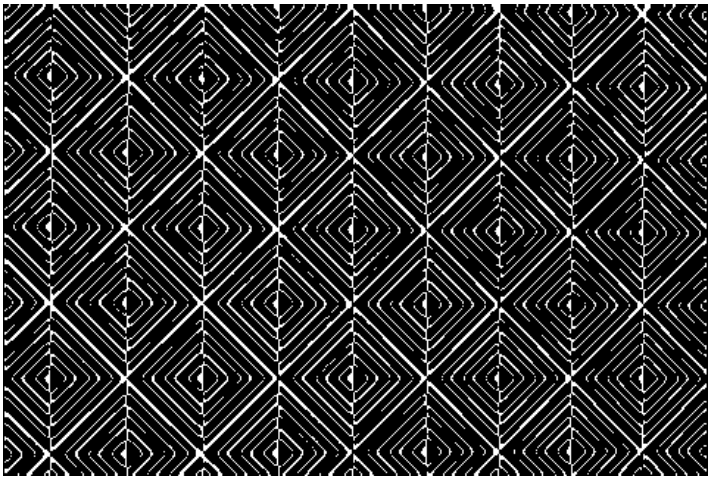

Рисунок 3 – Градиент по оси X

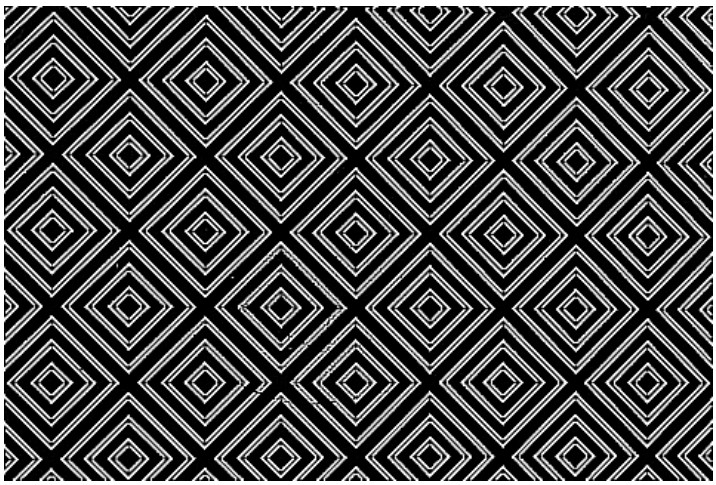

Рисунок 4 – Лапласиан

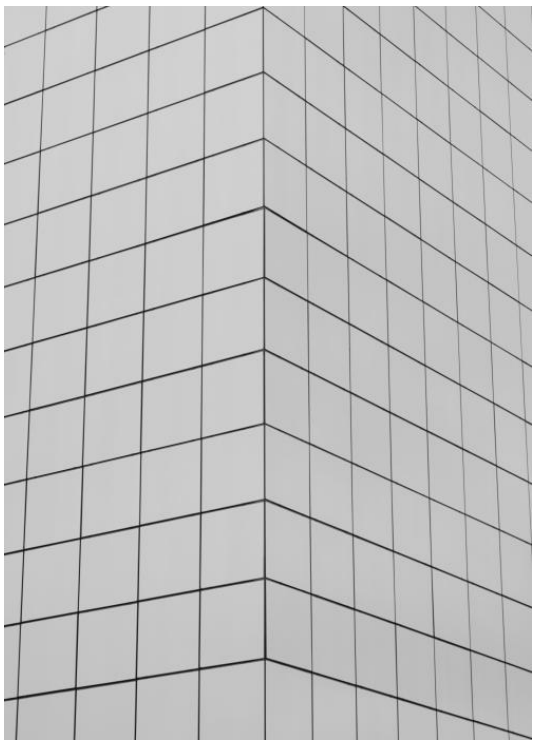

Рисунок 5 – Исходное изображение №2

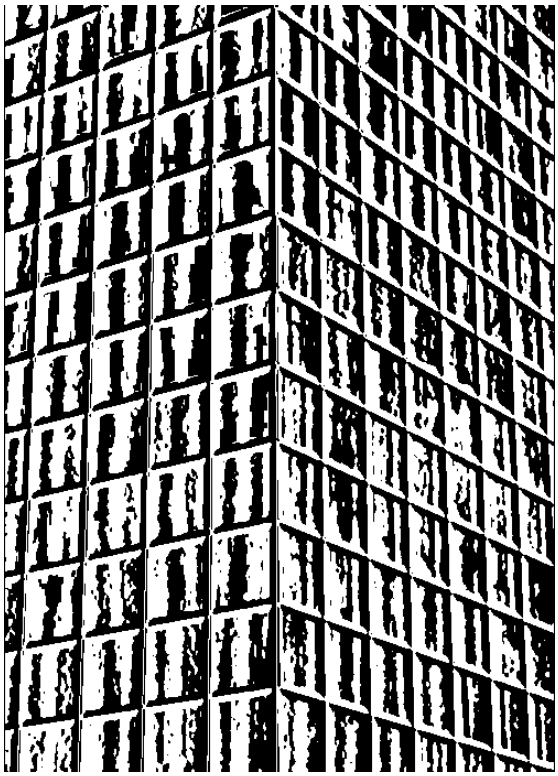

Рисунок 6 – Градиент по оси X

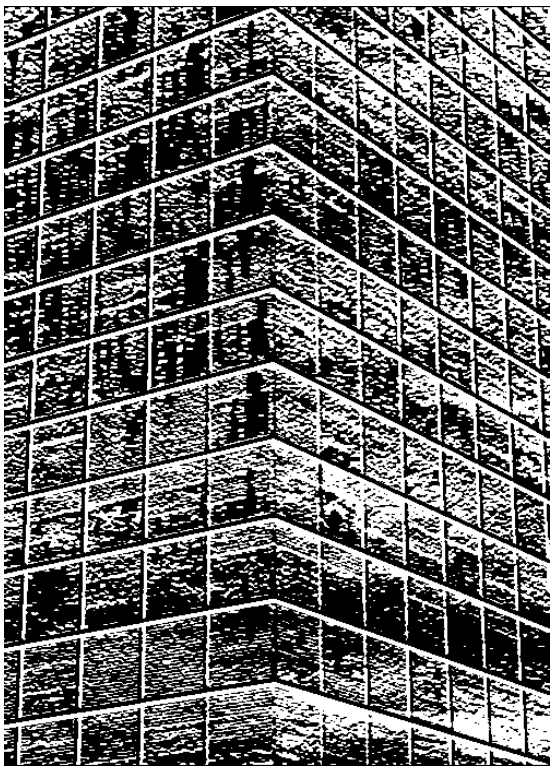

Рисунок 7 – Градиент по оси Y

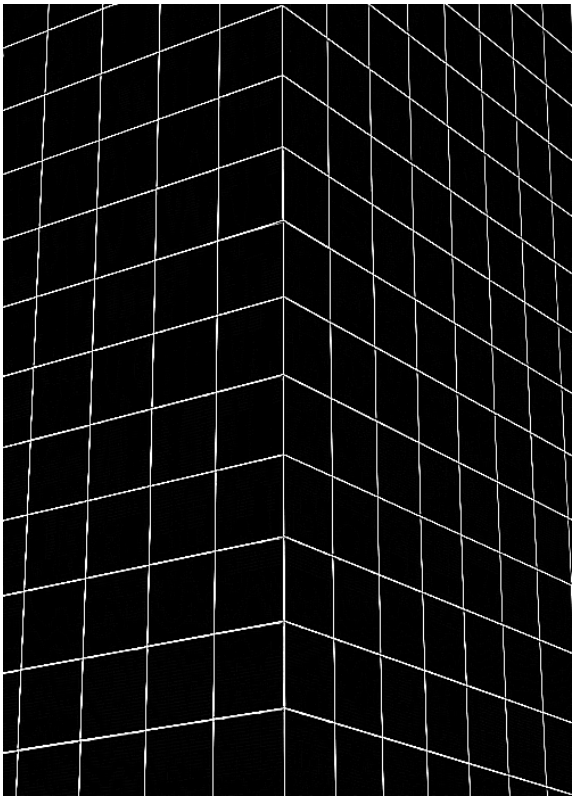

Рисунок 8 – Лапласиан

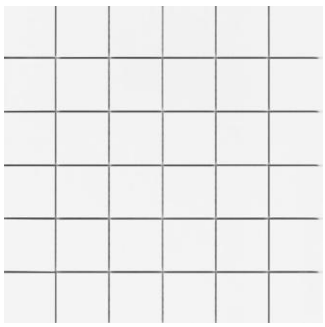

Рисунок 9 – Исходное изображение №3

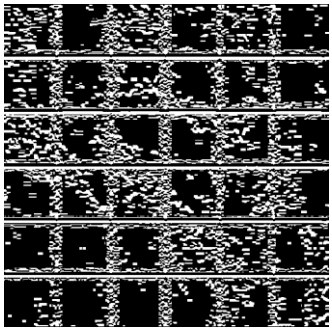

Рисунок 10 – Градиент по оси Y

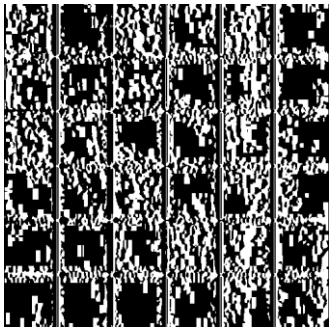

Рисунок 11 – Градиент по оси X

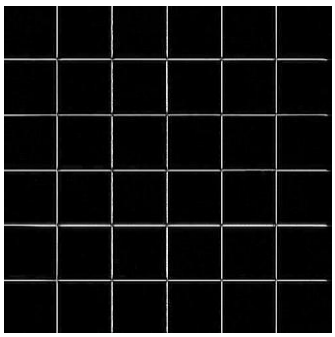

Рисунок 12 – Лапласиан

Задание №2

При применении напрямую к исходному изображению (рисунок 13) детектора границ Кенни, наблюдается сильный шум, вызванный неровностями (рисунок 14).

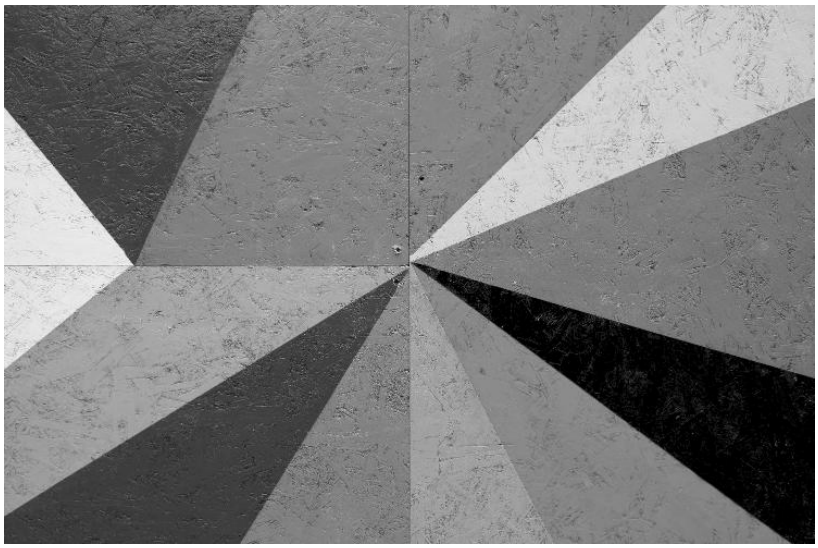

Рисунок 13 – Исходное изображение с чёткими границами

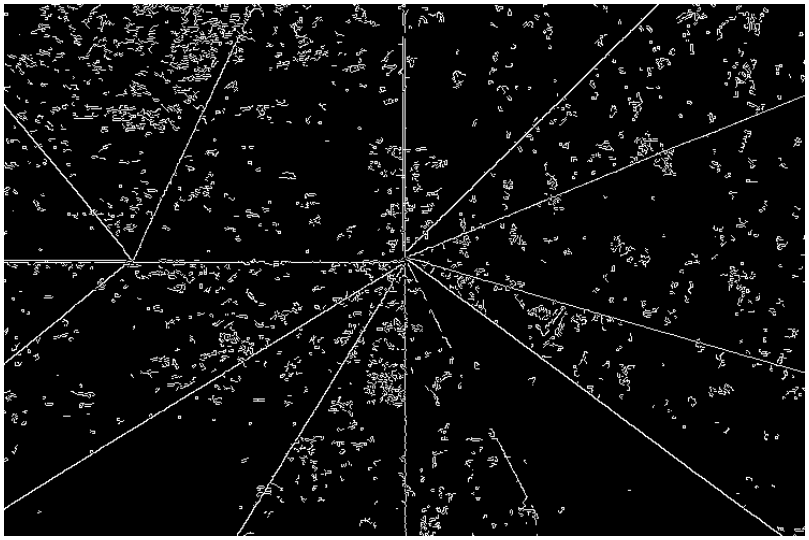

Рисунок 14 – Границы, найденные на исходном изображении

Для того, чтобы избавиться от этого шума необходимо, очевидно, использовать детектор границ на отфильтрованном изображении – рисунки 15 и 16.

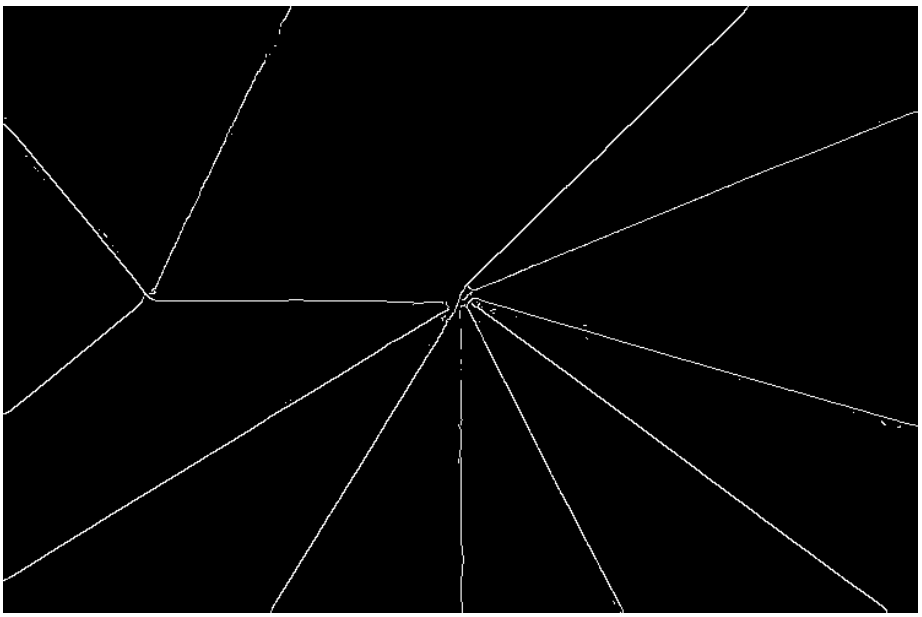

Рисунок 15 – Границы найденные на изображении после медианного фильтра

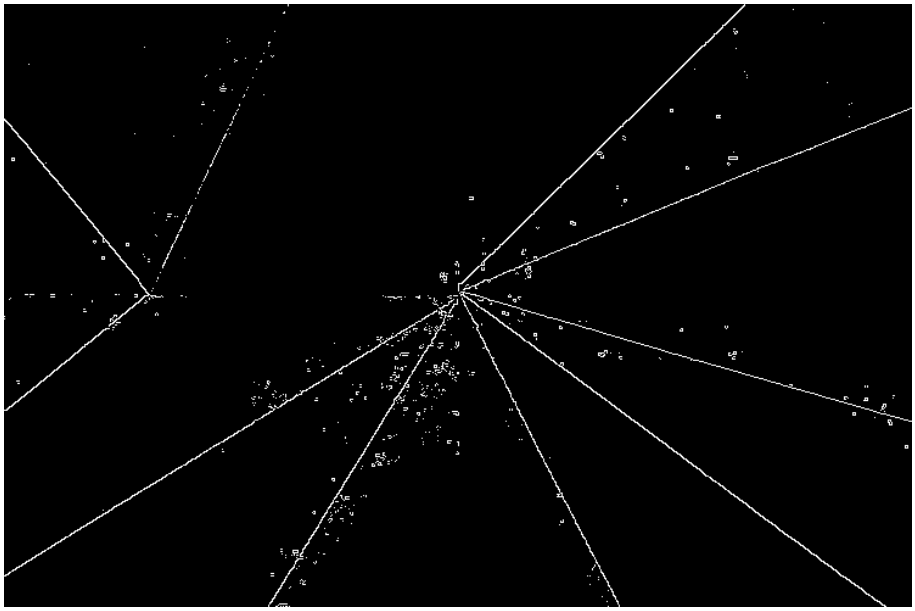

Рисунок 16 – Границы, найденные на изображении после двустороннего фильтра

Код для выполнения действий представлен ниже:

```
path = r"./3-4.png"
picture = cv2.imread(path, cv2.IMREAD_REDUCED_GRAYSCALE_2)
median = cv2.imread(r"./median.jpg",cv2.IMREAD_GRAYSCALE)
bilat = cv2.imread(r"./bilat.jpg",cv2.IMREAD_GRAYSCALE)
edges = cv2.Canny(picture, 100, 200)
median_edges = cv2.Canny(median, 36, 36)
bilat_edges = cv2.Canny(bilat, 100, 100)
```

```
cv2.imshow("median",median)
cv2.imshow("bilat",bilat)
cv2.imshow("3-4.png",picture)
cv2.imshow('canny', edges)
cv2.imshow('median_edges', median_edges)
cv2.imshow('bilat_edges', bilat_edges)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Для изображения с нечёткими границами (рисунок метод Кенни тоже работает довольно качественно – рисунок 18.

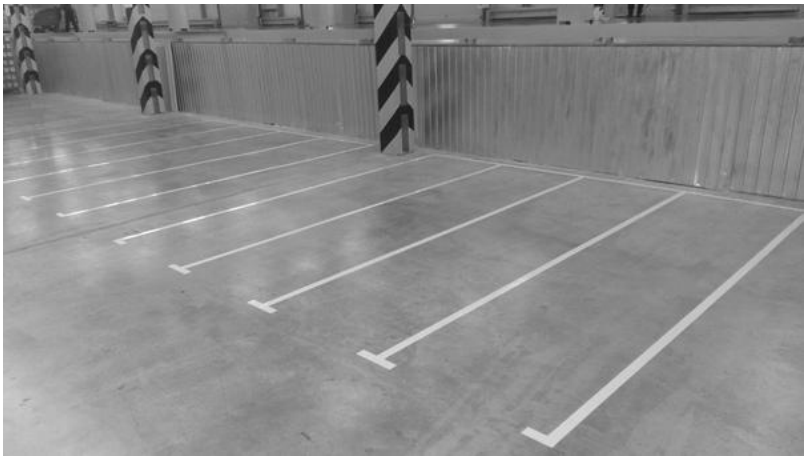

Рисунок 17 – Изображение с нечёткими границами

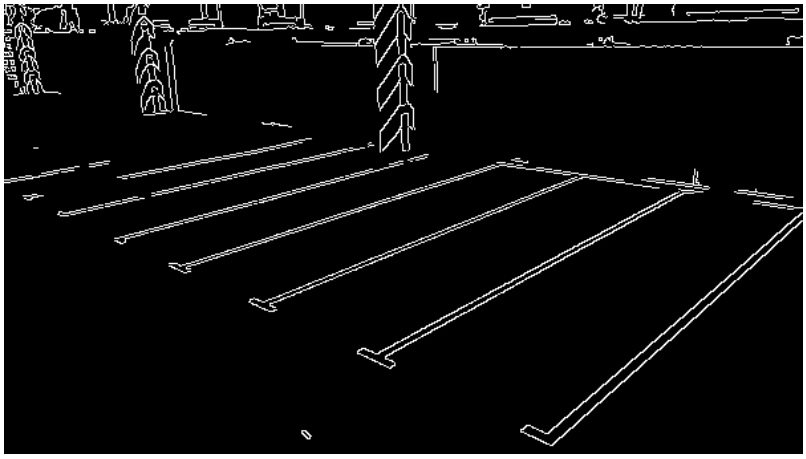

Рисунок 18 – Найденные границы на исходном изображении

Вывод

В ходе выполнения лабораторной работы были изучены способы детекции границ на изображениях. Оператор Собеля показал наименее

пригодный результат, однако он может быть полезен в тех случаях, когда необходимо найти границы лишь по одной из осей, тогда он производительнее чем лапласиан и метод Кенни. Последние, в свою очередь, показали хороший результат для поиска границ на «сложных изображениях».